

Introducción

Esta práctica consistió en usar el puerto GPIO de la Raspberry Pi para controlar una fila de siete LEDs mediante un arreglo Darlington ULN2003 y un display de siete segmentos mediante un decodificador BCD 74LS47, implementando por software una marquesina con velocidad variable, un efecto de rebote (ping-pong) y un efecto de encendido/apagado gradual mediante PWM, utilizando la biblioteca RPi.GPIO en Python.

Antecedentes

El GPIO de la Raspberry Pi expone pines de propósito general configurables como entradas o salidas digitales de 3.3V, con una corriente máxima por pin insuficiente para manejar varias cargas a la vez (Raspberry Pi Foundation, s.f.). El ULN2003A resuelve esto mediante un arreglo de transistores Darlington de colector abierto —dos transistores en cascada que multiplican la ganancia de corriente— capaz de conducir hasta 500mA por canal (Horowitz & Hill, 2015). El 74LS47 es un decodificador combinacional que traduce una entrada BCD de 4 bits al patrón de segmentos (a-g) correspondiente mediante una tabla de verdad fija, pensado para excitar directamente un display de ánodo común (Mano & Ciletti, 2018). La modulación por ancho de pulso (PWM) simula una salida analógica variando el ciclo de trabajo de una señal digital; RPi.GPIO la implementa por software, generándola desde el propio procesador en lugar de un periférico dedicado (Monk, 2016).

Desarrollo

El circuito se alambró conforme al manual: siete LEDs con una resistencia de 330Ω cada uno hacia las salidas del ULN2003, y el 74LS47 con sus salidas a-g hacia el display de siete segmentos, con LT y RBI a positivo. Antes de energizar se verificó con un multímetro la ausencia de continuidad entre los rieles de alimentación, y ya con una fuente de banco a 4.9V se probó cada entrada de ambos circuitos integrados para confirmar el mapeo antes de introducir la Raspberry Pi, que se conectó mediante jumpers directos a la cabecera GPIO en modo BOARD: los pines 12, 16, 18, 22, 24, 26 y 32 hacia IN1-IN7 del ULN2003, y los pines 36, 38, 40 y 37 hacia B, C, D y A del 74LS47.

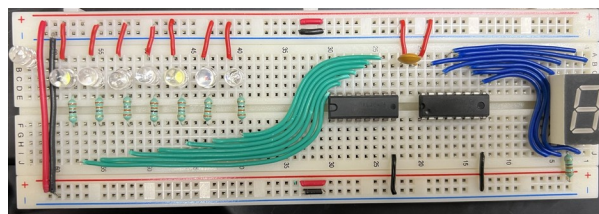


Figura 1. Circuito completo armado: LEDs y ULN2003 (izquierda), 74LS47 y display de siete segmentos (derecha).

El experimento 1 (parpadeo simultáneo de los siete LEDs) se resolvió pasando la lista completa de pines a una sola llamada de GPIO.output, en lugar de iterar sobre cada uno:

```
GPIO.output(pines_leds, GPIO.HIGH)
sleep(0.5)
GPIO.output(pines_leds, GPIO.LOW)
```

El experimento 2 (marquesina de izquierda a derecha) se resolvió recorriendo la lista de pines en su orden natural:

```
for pin in pines_leds:
```

```
GPIO.output(pin, GPIO.HIGH)
sleep(0.2)
GPIO.output(pin, GPIO.LOW)
```

El experimento 3 (marquesina de derecha a izquierda con velocidad definida por el usuario) se resolvió recorriendo los índices de la lista de pines en sentido inverso:

```
for i in range(len(pines_leds) - 1, -1, -1):
    GPIO.output(pines_leds[i], GPIO.HIGH)
    sleep(espera)
    GPIO.output(pines_leds[i], GPIO.LOW)
```

El experimento 4 (efecto de rebote) se resolvió concatenando un recorrido de ida (índices 0 a 6) con uno de vuelta (5 a 1), sin repetir los extremos, y recorriendo esa secuencia en un ciclo continuo:

```
ida = list(range(len(pines_leds)))
vuelta = list(range(len(pines_leds) - 2, 0, -1))
secuencia = ida + vuelta
```

El experimento 5 (encendido y apagado gradual) se resolvió incrementando el ciclo de trabajo de un objeto PWM en cien pasos distribuidos a lo largo de un segundo, manteniéndolo al 100% durante medio segundo, y reduciéndolo de igual forma:

```
for paso in range(1, PASOS + 1):
    duty = paso * INCREMENTO
    pwm.ChangeDutyCycle(duty)
    sleep(PAUSA)
```

Los cinco programas se ejecutaron sobre la Raspberry Pi y su comportamiento coincidió con lo esperado en cada caso.

Cuestionario

1. *¿Por qué usar corrimientos es la manera más eficiente de generar una marquesina?*

Usar corrimientos permite reutilizar una misma estructura de control —un índice que se desplaza sobre la lista de pines— en lugar de escribir una instrucción distinta para cada LED; si el corrimiento se implementa a nivel de bits, el procesador resuelve el desplazamiento en una sola operación de registro, reduciendo tanto el código como el tiempo por iteración frente a evaluar condicionales para cada salida (Mano & Ciletti, 2018).

2. *¿Qué ventajas o desventajas tiene usar un modulador de ancho de pulso sobre tiempos de espera programados (delays)?*

La ventaja del PWM sobre los delays es que la variación de brillo se logra conmutando a una frecuencia fija sin bloquear el hilo principal en múltiples tiempos de espera discretos, permitiendo un control continuo del nivel de potencia; su desventaja, al ser PWM por software en RPi.GPIO, es que la temporización depende de la carga del sistema operativo y puede presentar jitter que no ocurre con un generador de PWM por hardware (Monk, 2016).

3. *¿Sería posible generar una marquesina circular utilizando el 74LS47 y el display de siete segmentos? Justifique.*

No de forma directa: el 74LS47 decodifica un código BCD completo a un dígito fijo mediante una tabla de verdad interna, sin exponer control independiente de cada segmento, por lo que no puede usarse para desplazar un solo segmento encendido alrededor del display; esto requeriría manejar los segmentos a-g de forma directa desde el GPIO, sin el decodificador.

4. *¿Es posible configurar cualquier pin de la GPIO como PWM? Justifique.*

No. Únicamente los pines físicos asociados a los canales de PWM por hardware de la Raspberry Pi (GPIO12, GPIO13, GPIO18 y GPIO19 en numeración BCM) pueden generar la señal sin intervención continua del procesador; en el resto de los pines, RPi.GPIO simula el PWM por software, razón por la cual la práctica indica usar específicamente el pin físico 32 (GPIO12) para este experimento (Monk, 2016).

Conclusiones

Esta práctica permitió aplicar el concepto de interfaz de potencia entre un microcontrolador y una carga que excede su capacidad de corriente (ULN2003), el uso de un decodificador BCD dedicado en vez de manejar los segmentos individualmente (74LS47), y la diferencia entre variar el brillo mediante temporizaciones discretas o mediante PWM. La principal dificultad fue de verificación física —confirmar la polaridad de los LEDs y el mapeo de pines con el multímetro antes de energizar el circuito con la Raspberry Pi— más que de programación; como mejora, convendría sustituir los jumpers individuales por el cable listón original para reducir el riesgo de conexiones sueltas.

Bibliografía

- Horowitz, P., & Hill, W. (2015). The art of electronics (3.^a ed.). Cambridge University Press.
- Mano, M. M., & Ciletti, M. D. (2018). Digital design: With an introduction to the Verilog HDL, VHDL, and SystemVerilog (6.^a ed.). Pearson.
- Monk, S. (2016). Raspberry Pi cookbook (2.^a ed.). O'Reilly Media.
- Raspberry Pi Foundation. (s.f.). Raspberry Pi documentation — GPIO and the 40-pin header. Recuperado el 22 de septiembre de 2026, de <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#gpio-and-the-40-pin-header>
- Texas Instruments. (1988). SN54/74LS47, SN54/74LS48, SN54/74LS49 BCD-to-seven-segment decoders/drivers (hoja de datos). <https://www.ti.com/lit/ds/symlink/sn74ls47.pdf>
- Texas Instruments. (2016). ULN200x, ULQ200x high-voltage, high-current Darlington transistor arrays (hoja de datos, Rev. O). <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>